

Express

Programe 1 :

Yash vataliya-ce148

**Build simple authentication system with logging,
authentication and error handling middleware.**

authentication.js > authenticate

```
const express = require("express");
const app = express();

app.use((req, res, next) => {
  console.log(`${new Date().toISOString()} - ${req.method} ${req.url}`);
  next();
});

const authenticate = (req, res, next) => {
  const isLoggedIn = true;
  if (isLoggedIn) {
    console.log("User is authenticated");
    next();
  } else {
    res.status(401).send("Unauthorized");
  }
};

app.use(authenticate);

app.get("/", (req, res) => {
  res.send("Welcome to the home page!");
});

app.get("/profile", (req, res) => {
  res.send("Welcome to your profile!");
});
```

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send("Something went wrong!");
});

app.listen(3000, () => {
  console.log("Server is running on port 3000");
});
```

```
run `npm fund` for details
PS C:\Users\Aryan Vataliya\Desktop\CE sem 5\Advance Technology\Typescript\Project> node authentication.js
Server is running on port 3000
2023-08-27T16:58:36.592Z - GET /
User is authenticated
2023-08-27T16:58:36.667Z - GET /favicon.ico
User is authenticated
2023-08-27T16:58:42.447Z - GET /profile
User is authenticated
2023-08-27T16:58:49.298Z - GET /
User is authenticated
[]
```

← → ↻ ⓘ localhost:3000

Welcome to the home page!

← → ↻ ⓘ localhost:3000/profile

Welcome to your profile!

Explanation of execution:

- When a request comes in, it first goes through the logging middleware, which logs the request details.
- Then it moves to the authentication middleware which simulates user authentication.
- If authentication is successful the request proceeds to the appropriate route handler.
- If authentication fails a 401 unauthorized response is sent.
- If any errors occur during the request processing the error handling middleware catches and handles them by logging the error stack and sending a 500 internal server error response.

Programme 2 :

Neer patel-ce103

```
/**
 * The comments provide a detailed explanation of each section of the
 * code, including importing the module,
 * defining API endpoints, and starting the server. This code creates a
 * simple API using Express.js that can
 * retrieve a list of users or a specific user by ID.
 */
// Import the Express.js module
const express = require('express');
// Create an instance of the Express application
const app = express();
// Specify the port number the server will listen on
const port = 3000;
// Simulated database of users
const users = [
  { id: 1, name: 'Alice' },
  { id: 2, name: 'Bob' },
];
// Define API endpoint to get a list of all users
app.get('/api/users', (req, res) => {
  res.json(users); // Respond with the list of users as JSON
});
// Define API endpoint to get a specific user by ID
app.get('/api/users/:id', (req, res) => {
  // Extract the user ID from the URL parameter
  const userId = parseInt(req.params.id);
  // Find the user with the matching ID in the simulated database
  const user = users.find(user => user.id === userId);
  if (user) {
    // If the user is found, respond with the user details as JSON
    res.json(user);
  } else {
    // If the user is not found, respond with a 404 error and an error
    // message
    res.status(404).json({ error: 'User not found' });
  }
});
```

```

}
});
// Start the Express server and listen on the specified port
app.listen(port, () => {
// Display a message when the server starts
console.log(`API server listening at http://localhost:${port}`);
});

```

Programme 3 :

Vansh Shah-ce127

Create a simple Express.js application that simulates a basic task list. Write the necessary code to set up the application, define a route to render a task list using EJS templates, and start the server on port 3000. Explain the purpose of each part of the code and how it contributes to the functionality of the application.

Create the Files:

Create an index.js file for your Express application.

Create a views directory to store your EJS templates.

Inside the views directory, create an index.ejs file for rendering the task list.

```

index.js:
const express = require('express');
const app = express();
const PORT = 3000;
// Sample data - tasks
const tasks = [
  { id: 1, title: 'Buy groceries' },
  { id: 2, title: 'Finish homework' },
  { id: 3, title: 'Go for a run' }
];
// Set the view engine to EJS
app.set('view engine', 'ejs');
// Define a route to render the task list
app.get('/', (req, res) => {
res.render('index', { tasks });

```

```
});
// Start the server
app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});
```

views/index.ejs:

```
<!DOCTYPE html>
<html>
<head>
<title>Task List</title>
</head>
<body>
<h1>My Task List</h1>
<ul>
<% tasks.forEach(task => { %>
<li><%= task.title %></li>
<% }); %>
</ul>
</body>
</html>
```

Output:

```
My Task List
- Buy groceries
- Finish homework
- Go for a run
```

Programme 4 :

Dhruv Thakkar-ce140

Set up an Express.js server that listens on port 3000. Create an array called `tasks` to store task objects. Each task should have a unique `id` (integer) and a `name` (string). Implement a route to handle GET requests at `"/tasks"`. This route should respond with a JSON array containing all the tasks. Develop a route to handle POST requests at `"/tasks"`. This route should accept JSON data in the request body with a `name` property. Generate a unique `id` for the new task, add it to the `tasks` array, and respond with the created task's JSON data. Create a route at `"/tasks/:id"` to handle

DELETE requests. When a request is made to this route, find the task by id in the tasks array, remove it, and respond with a success message. Implement error handling middleware:

- a) For routes that don't exist, respond with a JSON object containing an error message and status code 404.
- b) For unhandled errors, respond with a JSON object containing an error message and status code 500.

```
const express = require('express');
const app = express();

const bodyParser = require('body-parser');

app.use(bodyParser.json());

const tasks = [];

// Error handling middleware for 404

app.use((req, res, next) => {

res.status(404).json({ error: 'Route not found' });

});
```

```
// Error handling middleware for unhandled errors
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Internal server error' });
});

// Get all tasks
app.get('/tasks', (req, res) => {
  res.json(tasks);
});

// Add a new task
app.post('/tasks', (req, res) => {
  const newTask = {
    id: tasks.length + 1,
    name: req.body.name
  };
  tasks.push(newTask);
  res.status(201).json(newTask);
});

// Delete a task by ID
app.delete('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const taskIndex = tasks.findIndex(task => task.id === taskId);

  if (taskIndex !== -1) {
    tasks.splice(taskIndex, 1);
    res.json({ message: `Task with ID ${taskId} deleted` });
  } else {
    res.status(404).json({ error: 'Task not found' });
  }
});

app.listen(3000, () => {
  console.log('Server is listening on port 3000');
});
```


Program 5 : Build a static weather app that provides the following features:

- Display a welcome message when users visit the root URL.
- Create a route that displays the current weather information for a specific city when users provide the city name in the URL.

Naitik Patel-ce102

```
const express = require('express'); const app = express();
const port = 3000;

app.get('/', (req, res) => { res.send('Welcome to the Weather App!');

});

// Display weather information for a specific city

app.get('/weather/:city', (req, res) => { const city = req.params.city;
const weatherData = {

city,
temperature: '25°C', condition: 'Sunny',

};

res.json(weatherData); });

// Start the server

app.listen(port, () => {
console.log(`Server is running on port ${port}`);
```

```
});
```

Programme 6 : Set timestamp using response in express js

Shivam Yadav-ce130

```
const express = require('express');
const app = express();
// Custom middleware to add a timestamp to response headers
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  res.setHeader('X-Timestamp', timestamp);
  next();
});
// Route handler
app.get('/', (req, res) => {
  res.send('Check the response headers for the timestamp.');
```